QuillAudits

AUDIT REPORT

November 2026

For

Tradevu

Table of Content

Executive Summary 04

Number of Security Issues per Severity 06

Summary of Issues 07

Checked Vulnerabilities 09

Techniques and Methods 11

Types of Severity 13

Types of Issues 14

Severity Matrix 15

Medium Severity Issues **16**

M-01 Changing the USDC address breaks per-offering accounting 16

M-02 Investors cannot recover an unresolved order without the issuer 18

M-03 One blacklisted investor blocks the issuer's whole refund batch 21

Low Severity Issues **23**

L-01 Any caller can take another investor's order ID first 23

L-02 Fees can be set as high as 100% 25

L-03 Renouncing ownership freezes the shared token logic 26

L-04 The supply cap ignores pending orders and can revert batch resolves 28

L-05 Gas: storage packing and a redundant mapping 30

L-06 Redemption rounding underpays the last redeemers 32

L-07 A fully drained offering can never be marked settled 34

L-08 getUserRedemptionAmount quotes payouts that will not be made 35

L-09 Unclaimed redemptions are swept to adminWallet with no grace period 36

L-10 Investors lose the rounding remainder of every order 37

L-11 A cancelled order makes the issuer's reject batch revert 38

Informational Severity Issues **40**

I-01 README points to a deployment helper that does not exist 40

I-02 The published interface does not match the escrow 41

I-03 depositUSDC reports misleading errors 42

I-04 README says the whole balance is burned on redemption 43

I-05 Events misreport amounts and parties 44

Threat Model 45

Automated Tests 110

Closing Summary & Disclaimer 111

Executive Summary

Project Name	Tradevu
Protocol Type	Defi - Trade Finance
Project URL	https://www.tradevu.co/
Overview	Tradevu protocol tokenizes trade-finance bill offerings settled in USDC. A protocol admin uses an upgradeable factory to create one ERC-20 bill-token proxy per offering; investors place USDC orders during the offering window, the issuer resolves them (sending principal to itself, fees to the admin wallet, and minting bill tokens), then funds redemption after maturity. The escrow tracks obligations separately per offering, lets token holders redeem during a fixed claim window, and sweeps unclaimed funds to the admin wallet afterward. AccessControl governs protocol, factory, issuer, and escrow-controller permissions; custom proxies provide upgrade authority, with token proxies resolving their shared logic implementation through an ImplementationAuthority.
Audit Scope	The scope of this audit was to analyze the Tradevu smart contracts for quality, security, and correctness.
Source Code link	github.com/Trade-Vu/capital-smart-contract
Contracts in Scope	TradevuEscrow.sol TradevuFactory.sol Token.sol ImplementationAuthority.sol] EscrowProxy.sol FactoryProxy.sol TokenProxy.sol IToken.sol ITradevuEscrow.sol ITradevuFactory.sol and IImplementationAuthority.sol
Branch	Main
Commit Hash	15e5faaf4da8c59f3649614ed7c5539a8536e4c0

Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Threat Modelling, Automated Testing, Functional Testing, Static Analysis
Review 1	30th September - 9th October, 2026
Updated Code Received	21st October, 2026
Review 2	29th November, 2026
Fixed In	https://github.com/Trade-Vu/capital-smart-contract/commit/fa52aa01ef7a7c5d988607614bc091a26ed1a501

Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	3 (16%)
Low	11 (58%)
Informational	5 (26%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	**4**	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	**3**	**7**	**5**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Changing the USDC address breaks per-offering accounting	Medium	Resolved
2	Investors cannot recover an unresolved order without the issuer	Medium	Resolved
3	One blacklisted investor blocks the issuer's whole refund batch	Medium	Resolved
4	Any caller can take another investor's order ID first	Low	Acknowledged
5	Fees can be set as high as 100%	Low	Resolved
6	Renouncing ownership freezes the shared token logic	Low	Resolved
7	The supply cap ignores pending orders and can revert batch resolves	Low	Acknowledged
8	Gas: storage packing and a redundant mapping	Low	Acknowledged
9	Redemption rounding underpays the last redeemers	Low	Resolved
10	A fully drained offering can never be marked settled	Low	Resolved

Issue No.	Issue Title	Severity	Status
11	getUserRedemptionAmount quotes payouts that will not be made	Low	Resolved
12	Unclaimed redemptions are swept to adminWallet with no grace period	Low	Acknowledged
13	Investors lose the rounding remainder of every order	Low	Resolved
14	A cancelled order makes the issuer's reject batch revert	Low	Resolved
15	README points to a deployment helper that does not exist	Informational	Resolved
16	The published interface does not match the escrow	Informational	Resolved
17	depositUSDC reports misleading errors	Informational	Resolved
18	README says the whole balance is burned on redemption	Informational	Resolved
19	Events misreport amounts and parties	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls
- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall

- ✓ Upgradeable safety
- ✓ Style guide violation
- ✓ Using throw
- ✓ Unsafe type inference
- ✓ Using inline assembly
- ✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Medium Severity Issues

M-01 Changing the USDC address breaks per-offering accounting

Resolved

Path

`TradevuEscrow.sol`

Function Name

`updateUSDC`

Description

The escrow tracks money in `orderPool[token]`, `settlementPool[token]` and `totalEscrowed`. Each is a plain number with no record of which USDC contract it is denominated in. `updateUSDC` swaps the `usdc` pointer and migrates nothing.

After the switch, every ledger still says the escrow owes the same number of units, but the units now mean a different token. The old balance sits in the old USDC contract with no code path to pay it out. Every later refund, redemption and sweep calls `usdc.safeTransfer` on the new token, so it pays from whatever the escrow holds of the new token. That balance belongs to offerings that were funded after the switch. A refund for a pre-switch order therefore either reverts (balance too small) or takes money from a post-switch offering.

```solidity
function updateUSDC(address usdc_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(usdc_ != address(0), "zero address");
    usdc = IERC20(usdc_);
}
```

Impact High

Pre-switch investors cannot be refunded or redeemed, because their money is in a token the escrow no longer touches. Post-switch investors can be short-changed when pre-switch claims are paid out of the new balance, so one offering's loss lands on another. The view `unaccountedBalance()` computes `usdc.balanceOf(this) - totalEscrowed` and reverts on underflow once the new balance is below `totalEscrowed`, which makes the solvency check unusable exactly when it is needed. The function emits no event, so the change is also invisible off-chain. The size of the loss is every unsettled balance at the moment of the switch.

M-01 Changing the USDC address breaks per-offering accounting

(continued)

Likelihood Low

Low. Only `DEFAULT_ADMIN_ROLE` can call it. The realistic trigger is an honest migration (a USDC contract change, or a typo in the new address) done while offerings are live, not an attack

POC

Owner calls updateUSDC without migrating/clearing earlier balances.

Recommendation

Remove the function. If a migration path is required, allow it only when nothing is owed, check that the new token matches, and emit an event:

```
event USDCUpdated(address indexed oldUsdc, address indexed newUsdc);

function updateUSDC(address usdc_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(usdc_ != address(0), "zero address");
    require(totalEscrowed == 0, "funds held");
    require(
        IERC20Metadata(usdc_).decimals() == IERC20Metadata(address(usdc)).decimals(),
        "decimals differ"
    );
    emit USDCUpdated(address(usdc), usdc_);
    usdc = IERC20(usdc_);
}
```

Put the admin behind a multisig and a timelock, so a switch is announced before it takes effect.

> **Tradevu Team's Comment**
>
> *Applied the recommended fix.*

Medium Severity Issues

M-02 Investors cannot recover an unresolved order without the issuer

Resolved

Path

`TradevuEscrow.sol`

Function Name

`rejectOrder(), resolveOrder()`

Description

A placed order sits in `orderPool` until the issuer resolves or rejects it but only a holder of `ISSUE_ROLE` on that token can do either.

`resolveOrder` also stops at maturity (`if (block.timestamp > maturityDate) revert OfferingEnded();`), so after maturity the only way out of an unresolved order is the issuer calling `rejectOrder` , which has no deadline but needs the role. The token's `DEFAULT_ADMIN_ROLE` can revoke the issuer's role, and then `resolveOrder` , `rejectOrder` , `depositUSDC` and `withdrawRemainingUSDC` are uncallable for that offering until the admin grants the role again. The same lock applies if the issuer loses its key or simply stops responding. `withdrawRemainingUSDC` only sweeps `settlementPool` , so the money in `orderPool` is not swept either. It stays in the escrow.

```solidity
bytes32 ISSUE_ROLE = keccak256("escrow.issuer.tradevu");
if (!AccessControlUpgradeable(order.token).hasRole(ISSUE_ROLE, msg.sender)) {
    revert AccessControlUnauthorizedAccount(msg.sender, ISSUE_ROLE);
}
```

Impact High

Each affected investor's principal plus fee stays locked with no on-chain recourse. Per order that is up to `maxInvestment` plus the fee; across an offering, up to the whole `orderPool` . The investor receives nothing: no tokens, no refund. The README documents the issuer's role in settlement, but not the case where the issuer is absent.

Likelihood Low

Low. It needs an unresponsive or removed issuer, a lost key, or an admin revoking the role. Because the loss is permanent for the investor, it is worth removing the dependency.

M-02 Investors cannot recover an unresolved order without the issuer

(continued)

POC

```
it("Finding 2: an unresolved order cannot be recovered by its owner, and the admin can lock it
for good", async function () {
    const s = await setup();
    const before = await s.usdc.balanceOf(s.user1.address);

    // 1. user1 invests. The issuer never acts on the order.
    await s.escrow.connect(s.user1).placeOrder(s.o1.target, ORDER, "order-1");
    expect(await s.usdc.balanceOf(s.user1.address)).to.equal(before - PAID);

    // 2. Maturity passes. The issuer can no longer resolve, and the investor cannot refund
themselves.
    await time.increase(1000);
    await expect(s.escrow.connect(s.issuer).resolveOrder(["order-1"]))
      .to.be.revertedWithCustomError(s.escrow, "OfferingEnded");
    await expect(s.escrow.connect(s.user1).rejectOrder(["order-1"]))
      .to.be.revertedWithCustomError(s.escrow, "AccessControlUnauthorizedAccount");

    // 3. The token admin revokes the issuer role (or the issuer key is lost). Nobody can refund
the order.
    await s.o1.connect(s.admin).revokeRole(await s.o1.ISSUE_ROLE(), s.issuer.address);
    await expect(s.escrow.connect(s.issuer).rejectOrder(["order-1"]))
      .to.be.revertedWithCustomError(s.escrow, "AccessControlUnauthorizedAccount");
    await expect(s.escrow.connect(s.issuer).depositUSDC("o1", 1n)).to.be.reverted;
    await expect(s.escrow.connect(s.issuer).withdrawRemainingUSDC("o1")).to.be.reverted;

    // 4. The money is still in the escrow, booked to the offering, with no path out for the
investor.
    expect(await s.escrow.orderPool(s.o1.target)).to.equal(PAID);
    expect(await s.usdc.balanceOf(s.escrow.target)).to.equal(PAID);
    expect(await s.usdc.balanceOf(s.user1.address)).to.equal(before - PAID);
    expect(await s.o1.balanceOf(s.user1.address)).to.equal(0n);  // no tokens either
  });
```

Recommendation

Add an investor-callable cancel for unresolved orders after maturity. It pays only the order's owner, so it cannot be abused. Resolve is already blocked after maturity, so the two paths cannot overlap.

```
function cancelOrder(string calldata actionId) external nonReentrant {
    Order storage order = orders[actionId];
    if (order.token == address(0)) revert InvalidOrder();
    if (order.resolved) revert OrderAlreadyResolved();
    if (order.user != msg.sender) revert NotOrderOwner();

    (, , , uint256 maturityDate) = IToken(order.token).getAmountsAndDate();
    if (block.timestamp <= maturityDate + CANCEL_GRACE) revert OfferingNotEnded();
```

M-02 Investors cannot recover an unresolved order without the issuer

(continued)

```
    order.resolved = true;
    uint256 fee = (order.amount * IToken(order.token).getBillDetails().investmentFee) / 10000;
    uint256 refund = order.amount + fee;

    orderPool[order.token] -= refund;
    totalEscrowed -= refund;
    usdc.safeTransfer(order.user, refund);
}
```

`CANCEL_GRACE` (for example 7 days) gives the issuer time to reject first. Mirror whatever `rejectOrder` emits so indexers see both. Separately, consider making `ISSUE_ROLE` revocable only through a timelock.

Tradevu Team's Comment

Fixed. Applied recommendation.

Medium Severity Issues

M-03 One blacklisted investor blocks the issuer's whole refund batch

Resolved

Path

`TradevuEscrow`

Function Name

`rejectOrder(), resolveOrder()`

Description

rejectOrder and resolveOrder process up to 100 orders in one transaction and pay each recipient directly inside the loop: the refund to the investor in rejectOrder (L260), and the principal to the calling issuer in resolveOrder (L368). USDC reverts any transfer to or from a blacklisted address.

If one investor in a refund batch is blacklisted, their transfer reverts, and so does the whole transaction: nobody in the batch is refunded. A blacklisted issuer address cannot resolve any order, because the principal is always paid to the caller. A USDC-wide pause stops every flow at once.

The issuer has to identify the failing order and resubmit the batch without it. The blacklisted investor's own order cannot be refunded until the blacklisting is lifted.

Impact Medium

Low. Other investors' refunds and settlements are delayed until the issuer rebuilds the batch, but no funds are lost. The blacklisted investor's funds stay in the escrow, though USDC would freeze them anyway.

Likelihood Medium

Low. Requires USDC's issuer to blacklist a participant.

M-03 One blacklisted investor blocks the issuer's whole refund batch

(continued)

POC

`it("the batch reverts and nobody in it is refunded until the failing order is removed", async function () {
const ctx = await loadFixture(deployProtocol);
const offering = await createOffering(ctx, "bill-1");
await time.increaseTo(offering.t.startDate);
await placeOrder(ctx, offering, ctx.alice, USDC(500), "alice-1");
await placeOrder(ctx, offering, ctx.bob, USDC(500), "bob-1");
await placeOrder(ctx, offering, ctx.carol, USDC(500), "carol-1");

await ctx.usdc.blacklist(ctx.alice.address, true); // USDC's issuer blacklists one investor

await expect(ctx.escrow.connect(ctx.issuer).rejectOrder(["alice-1", "bob-1", "carol-1"]))
.to.be.revertedWithCustomError(ctx.usdc, "Blacklisted").withArgs(ctx.alice.address);
expect(await ctx.usdc.balanceOf(ctx.bob.address)).to.equal(0n);
expect(await ctx.usdc.balanceOf(ctx.carol.address)).to.equal(0n);

// Only a rebuilt batch without Alice's order goes through.
await ctx.escrow.connect(ctx.issuer).rejectOrder(["bob-1", "carol-1"]);
expect(await ctx.usdc.balanceOf(ctx.bob.address)).to.equal(USDC(510));
});`

Recommendation

Isolate failures per recipient in batch operations, for example by recording failed payouts for a later retry, or by letting recipients withdraw what they are owed instead of pushing payments inside the loop.

> **Tradevu Team's Comment**
>
> *Fixed as per recommendation.*

Low Severity Issues

L-01 Any caller can take another investor's order ID first

Acknowledged

Path

`TradevuEscrow.sol`

Function Name

`placeOrder()`

Description

Both the used-ID map and the order map are keyed by a string the caller picks.

The ID is not tied to the caller. An attacker who learns an ID before the victim's transaction lands places a minimum order with it. The victim's call then reverts with `ActionIdAlreadyUsed`, and the order under that ID belongs to the attacker. The ID can leak from a public mempool (the victim's pending `placeOrder` calldata), from the backend's API, or from a predictable ID scheme.

```solidity
if (actionIdUsed[actionID]) revert ActionIdAlreadyUsed();
...
orders[actionID] = Order({ token: token, user: msg.sender, amount: amount, resolved: false });
actionIdUsed[actionID] = true;
```

Impact Low

The victim does not lose funds, because their transaction reverts, and can retry with a fresh ID. The harm is denied service for a chosen investor.

Likelihood Medium

Medium, if IDs are guessable or visible before inclusion.

Recommendation

Make the stored key depend on the caller, so two users can never collide:

```
mapping(bytes32 => Order) public orders;

function placeOrder(address token, uint256 amount, bytes32 id) external nonReentrant {
    bytes32 key = keccak256(abi.encode(msg.sender, id));
    if (orders[key].token != address(0)) revert ActionIdAlreadyUsed();
```

L-01 Any caller can take another investor's order ID first *(continued)*

```
    ...
    emit OrderPlaced(token, msg.sender, amount, id, key);
}
```

The issuer's `resolveOrder` and `rejectOrder` then take the keys (`bytes32[]`), which the `OrderPlaced` event provides. An alternative is an on-chain counter (`orderId = ++nextOrderId`) with the caller's reference stored as metadata only.

Tradevu Team's Comment

Id is generated at backend level which is unique and can be known to outside world.

**cant be known to outside world. They may see in mempool but that is not worth.*

Low Severity Issues

L-02 Fees can be set as high as 100%

Resolved

Path

`Token.sol`

Function Name

`initialise()`

Description

The fee checks stop only values that would underflow the escrow so any fee up to 10,000 basis points (100%) is accepted. They are fixed at deployment and cannot be changed afterwards.

```solidity
if (billDetails_.investmentFee > 10000) revert InvalidFee(billDetails_.investmentFee);
if (billDetails_.distributionFee > 10000) revert InvalidFee(billDetails_.distributionFee);
```

Impact Medium

At a 100% `investmentFee`, `placeOrder` charges `amount + amount`, so the investor pays double and the fee goes to `adminWallet` on resolve. At a 100% `distributionFee`, `depositUSDC` sends the whole deposit to `adminWallet` as a fee and sets `settlementPool` to zero. Redeeming then burns tokens for nothing, because the pool is empty.

Likelihood Low

Recommendation

Cap each fee at a sensible maximum, and surface the terms in the offering's events.

> **Tradevu Team's Comment**
>
> *token's initialize already checks for fee > 2000 bps*

Low Severity Issues

L-03 Renouncing ownership freezes the shared token logic

Resolved

Path

`Renouncing ownership freezes the shared token logic`

Description

Every `TokenProxy` reads its logic address from one shared `ImplementationAuthority` and only the owner can change it.

`Ownable` exposes `renounceOwnership()` which sets the owner to the zero address. After that no one can call `updateImplementation`, and the logic every token resolves to is frozen.

Ownership can also be lost by a transfer to a wrong address, because `transferOwnership` takes effect immediately.

```
```solidity
contract ImplementationAuthority is IImplementationAuthority, Ownable {
    function updateImplementation(address _newImplementation) external onlyOwner { ... }
}
```

Impact Medium

A bug found later in the token logic cannot be fixed through the authority. Funds are not at risk from the renounce itself, and a token that is working keeps working.

Likelihood Low

Recommendation

1. Keep the owner as a multisig behind a timelock.
2. If the logic is meant to stay upgradeable, disable the renounce and use a two-step transfer:

```
contract ImplementationAuthority is IImplementationAuthority, Ownable2Step {
    function renounceOwnership() public pure override {
        revert("renounce disabled");
    }
    ...
}
```
```

L-03 Renouncing ownership freezes the shared token logic *(continued)*

Tradevu Team's Comment

Disabled the renounce ownership functionality.

Low Severity Issues

L-04 The supply cap ignores pending orders and can revert batch resolves

Acknowledged

Path

`TradevuEscrow.sol`

Function Name

`placeOrder(), resolveOrder()`

Description

`placeOrder` checks the cap against minted supply only.

Tokens are minted only at `resolveOrder`, so `totalSupply()` does not move while orders are pending. Any number of orders can be accepted whose combined tokens exceed the cap. In a live test, two 1,500 USDC orders (300 tokens each, cap 400) were both accepted. `resolveOrder` repeats the cap check inside its loop, so the first order that would pass the cap reverts the entire batch.

```
```solidity
uint256 currentTotalSupply = IERC20(token).totalSupply();
if (currentTotalSupply + tokensToMint > maxTotalSupply) {
    ...
    revert TotalSupplyExceeded(tokensToMint, availableTokens);
}
```

Impact Low

Orders beyond the cap hold investor funds in `orderPool` until the issuer rejects them. One unfillable ID in a batch of up to 100 blocks the other 99 until the issuer finds and removes it. No funds are lost; the cost is delay, extra issuer work and confusion.

Likelihood Medium

edium. Any popular offering that approaches its cap hits this, and no attacker is needed.

Recommendation

Count tokens that are accepted but not yet minted, and release them on resolve and reject:

```
mapping(address => uint256) public pendingTokens;

// placeOrder
```
```

L-04 The supply cap ignores pending orders and can revert batch resolves *(continued)*

```
if (currentTotalSupply + pendingTokens[token] + tokensToMint > maxTotalSupply) {
    revert TotalSupplyExceeded(tokensToMint, availableTokens);
}
pendingTokens[token] += tokensToMint;

// resolveOrder and rejectOrder, per order
pendingTokens[order.token] -= tokensToMint;
```

Also, consider allowing `resolveOrder` skip an over-cap order instead of reverting the batch.

Tradevu Team's Comment

Acknowledged. Orders are accepted and issuer will resolve and reject them.

Low Severity Issues

L-05 Gas: storage packing and a redundant mapping

Acknowledged

Path

`TradevuEscrow.sol`

Function Name

`placeOrder(), resolveOrder()`

Description

`Order` uses four storage slots (`token` , `user` , `amount` , `resolved`), and `actionIdUsed` duplicates what `orders[id].token != address(0)` already says.

Each fresh slot costs a 22,100 gas storage write.

```solidity
struct Order {
    address token;   // slot 0
    address user;    // slot 1
    uint256 amount;  // slot 2
    bool resolved;   // slot 3
}
mapping(string => bool) public actionIdUsed;   // one more slot per order
```

Impact Low

Measured on Hardhat with the repo's optimizer settings, 36-character IDs, a 40-order lifecycle, identical final balances in every variant: `placeOrder` averaged 228,831 gas before and 180,738 after packing `Order` into two slots and dropping `actionIdUsed` , a saving of about 48k (21%) per order. `resolveOrder` per order in a 20-batch fell from 69,512 to 46,460.

Moving IDs from `string` to `bytes32` saves only about 1.8k of execution, but cuts calldata by 49% to 73%, which matters on Base where the L1 data fee grows with calldata.

Likelihood Low

L-05 Gas: storage packing and a redundant mapping *(continued)*

Recommendation

```
struct Order {
    address token;    // 20 bytes
    bool resolved;    //  1 byte  -> slot 0
    address user;     // 20 bytes
    uint96 amount;    // 12 bytes -> slot 1 (up to 7.9e28 base units)
}

// duplicate check without actionIdUsed
if (orders[key].token != address(0)) revert ActionIdAlreadyUsed();
```

Check that `maxInvestment` always fits in `uint96` before narrowing. Use `bytes32` IDs (finding 3), cache the per-token reads in the batch loops, and send one USDC transfer per batch. These patches change the ABI (`orders()` field order, no `actionIdUsed`).

> **Tradevu Team's Comment**
>
> *Acknowledged. Explicit mapping is there for backend in case they need to check actionid.*

Low Severity Issues

L-06 Redemption rounding underpays the last redeemers

Resolved

Path

`TradevuEscrow.sol`

Description

When redemption is funded, the distribution fee is taken once on the whole deposit, rounded down, and the remainder becomes the pool. At redemption the fee is taken again on each holder's share, also rounded down. Each per-holder rounding leaves up to one USDC base unit of fee uncollected, so total payouts can exceed the pool by up to one base unit per holder.

Redemption is first come, first served. When the pool runs out, the next holder is paid whatever is left, has only a proportional part of their tokens burned, and keeps the rest. Those leftover tokens cannot be redeemed for anything, because the pool is empty.

Separately, a holder whose gross share rounds down to zero (trueBillPrice × balance 0

```
await expect(ctx.escrow.connect(ctx.carol).redeemBillTokens("bill-1"))
.to.changeTokenBalance(ctx.usdc, ctx.carol, 0n);
expect(await offering.token.balanceOf(ctx.carol.address)).to.equal(0n);
});
`
```

Impact Low

Low. The amounts are rounding dust, at most one USDC base unit per holder, but they fall entirely on the last redeemers instead of being shared.

Likelihood High

High. It happens naturally whenever holders' shares do not divide evenly by the fee rate. A holder could deliberately split balances to create it, but that costs far more in gas than it shifts.

POC

```
`it("ten small holders leave the pool 10 base units short and the last redeemer absorbs it", async function () {
const { ctx, offering } = await loadFixture(shortPool);
const owed = await ctx.escrow.getUserRedemptionAmount("bill-1", ctx.alice.address);
const pool = await ctx.escrow.settlementPool(offering.token.target);
expect(owed - pool).to.equal(10n); // one base unit per small holder
```

L-06 Redemption rounding underpays the last redeemers *(continued)*

```
await expect(ctx.escrow.connect(ctx.alice).redeemBillTokens("bill-1"))
.to.changeTokenBalance(ctx.usdc, ctx.alice, pool); // paid 10 base units less than owed
expect(await offering.token.balanceOf(ctx.alice.address)).to.be.gt(0n); // leftover tokens
expect(await ctx.escrow.settlementPool(offering.token.target)).to.equal(0n); // worth nothing now
});

it("a holder whose share rounds down to zero has the whole balance burned for nothing", async function
() {
const { ctx, offering } = await loadFixture(soldOutAndFunded);
await offering.token.connect(ctx.alice).transfer(ctx.carol.address, 100_000_000_000n); // share 0.625 -
> 0

await expect(ctx.escrow.connect(ctx.carol).redeemBillTokens("bill-1"))
.to.changeTokenBalance(ctx.usdc, ctx.carol, 0n);
expect(await offering.token.balanceOf(ctx.carol.address)).to.equal(0n);
});
`
```

Recommendation

Round per-holder fees in the protocol's favour so total payouts can never exceed the pool, and never burn a balance whose payout rounds to zero.

> **Tradevu Team's Comment**
>
> *Fixed*

Low Severity Issues

L-07 A fully drained offering can never be marked settled

Resolved

Path

`TradevuEscrow.sol`

Function Name

`withdrawRemainingUSDC()`

Description

withdrawRemainingUSDC sets isWithdrawn, zeroes the pool, and then reverts with NoUSDCLeft if there was nothing to send, which undoes the flag. Once a pool has been paid out completely (after a capped redemption as in issue 6, or with a 100% distribution fee), the sweep can never succeed and isWithdrawn stays false for good. The README says the sweep "marks the offering settled".

Impact

Low

Low. No funds are involved, but the offering can never be closed on-chain. Monitoring and back-office systems that wait for isWithdrawn or RemainingUSDCWithdrawn see it as unsettled indefinitely, and every sweep attempt reverts with an error that reads like a fault.

Likelihood

Medium

Medium. Happens whenever redemptions drain the pool completely.

Recommendation

Allow an offering whose pool is empty to be closed (marked settled without a transfer), so the documented end state is always reachable.

> **Tradevu Team's Comment**
>
> *Applied the recommended fix.*

Low Severity Issues

L-08 getUserRedemptionAmount quotes payouts that will not be made

Resolved

Path

`TradevuEscrow.sol`

Function Name

`getUserRedemptionAmount()`

Description

getUserRedemptionAmount computes a holder's share and fee but ignores the pool balance, the claim window and whether the offering has been swept. It keeps quoting the full payout after the window has closed and the pool has been swept, when redemption can only revert. When the pool is short (issue 6), it quotes more than redeemBillTokens pays. Before funding it returns 0, which is indistinguishable from "nothing owed".

Impact Low

Low. Front ends and integrations that rely on the quote show holders amounts they cannot receive.

Likelihood Medium

Recommendation

Make the quote reflect what redemption would actually pay at that moment, including the pool balance, the claim window and settlement state, or document clearly that it is only an upper bound.

> **Tradevu Team's Comment**
>
> *Fixed.*

Low Severity Issues

L-09 Unclaimed redemptions are swept to adminWallet with no grace period

Acknowledged

Path

`TradevuEscrow`

Function Name

`claimWindow`

Description

The claim window opens when the issuer funds redemption and lasts claimWindow seconds. Once it ends, redeemBillTokens reverts with ClaimTimeEnded, and the issuer can sweep everything left in the pool to adminWallet. There is no grace period, no extension and no later claim path.

A holder who does not redeem in time loses the entire payout, and it becomes protocol revenue instead of remaining claimable. Holders who acquired tokens by transfer may not know when funding happened, and Token.initialize only requires claimWindow > 0, so a window can be as short as one second.

Impact Medium

Medium for the affected holders: their entire redemption value moves to adminWallet.

Likelihood Low

Low. A holder has to miss the window; this becomes more likely with short windows or with holders who received tokens by transfer.

Recommendation

Guarantee holders a reasonable minimum claim period and some way to claim after it closes, for example by keeping unclaimed balances in a reserve holders can still draw from, rather than converting them into protocol revenue.

> **Tradevu Team's Comment**
>
> *Acknowledged. User has to redeem till claim window time.*

Low Severity Issues

L-10 Investors lose the rounding remainder of every order

Resolved

Path

`TradevuEscrow.sol`

Description

An order's allocation is amount × 10^decimals / issuePrice, rounded down, but the issuer receives the full amount, and redemption pays only for the tokens actually minted. The remainder, worth less than one token unit per order, goes to the issuer with no matching entitlement for the investor, who also pays the investment fee on it.

At 18 decimals this is negligible, but with few decimals it becomes material. With a 0-decimal token priced at 5 USDC, a 102 USDC order mints 20 tokens (100 USDC worth), and the extra 2 USDC is lost to the investor.

Impact Low

Low. Negligible with high token decimals; up to one token's price per order with low decimals.

Likelihood High

Recommendation

Only accept order amounts that convert exactly into token units, or return the unallocated remainder to the investor.

> **Tradevu Team's Comment**
>
> *Applied the first recommendation i.e. Only accept order amounts that convert exactly into token units.*

Low Severity Issues

L-11 A cancelled order makes the issuer's reject batch revert

Resolved

Path

`TradevuEscrow.sol`

Function Name

`rejectOrder(), cancelOrder()`

Description

In the fix review at commit (`b74b0ae`), we noted that the new function `cancelOrder` marks the order resolved meanwhile rejectOrder in a batch loop reverts on a resolved order:

```
// cancelOrder
order.resolved = true;

// rejectOrder, inside the batch loop
if (order.resolved) revert OrderAlreadyResolved();
```

An investor can cancel once maturityDate + CANCEL_GRACE has passed. If the issuer's batch of up to 100 IDs includes an order the investor cancels first, the whole batch reverts.

Impact

The issuer has to find the cancelled ID and resubmit without it. Other investors' refunds are delayed until then. No funds are lost, and the investor who cancelled gets their money back.

Likelihood

Medium

Medium once the grace period has passed with unresolved orders; low before.

POC

1. Two investors place 500 USDC orders an and b. Time moves past maturity plus 7 days.
2. The investor of a calls cancelOrder("a").
3. The issuer calls rejectOrder(["a","b"]). It reverts with OrderAlreadyResolved.
4. The issuer calls rejectOrder(["b"]). It succeeds.

L-11 A cancelled order makes the issuer's reject batch revert *(continued)*

Recommendation

Use the order's own amount as the marker, with no new variable. cancelOrder already holds the amount in a local, so it zeroes the stored one after setting the resolved flag, and rejectOrder skips an order that is resolved and has zero amount:

```
// cancelOrder, after order.resolved = true;
order.amount = 0;

// rejectOrder, replacing the OrderAlreadyResolved revert in the loop
if (order.resolved) {
    if (order.amount == 0) continue;   // cancelled by its owner: nothing left to refund
    revert OrderAlreadyResolved();
}
```

Tradevu Team's Comment

Fixed.

Informational Severity Issues

I-01 README points to a deployment helper that does not exist

Resolved

Path

```
README points to a deployment helper that does not exist
```

Description

The README says the initData passed to deployToken is ABI-encoded off-chain, and links to script/encodeData.js. That file is not in the repository; script/ contains only deploy.js. Integrators therefore have no reference for building correct Token.initialize data, which the factory executes without validating it.

Recommendation

Add the referenced helper or document the encoding in the README.

> **Tradevu Team's Comment**
>
> *Fixed*

Informational Severity Issues

I-02 The published interface does not match the escrow

Resolved

Path

`ITradevuEscrow.sol, IToken.sol, TradevuEscrow.sol`

Description

ITradevuEscrow.OrderDetails lists user before token and adds an orderRedeemed flag. getOrder, however, returns (token, user, amount, resolved), and no redeemed flag exists anywhere. Both fields are addresses, so a consumer that decodes getOrder using the interface struct's field order silently swaps them.

The interfaces also declare errors that are never used. On the escrow: OrderAlreadyRedeemed, UserAmountMismatch and MoreThanMaxPerUser. On the token: NotTransferrable, DiscountBillAmountOvershoot and MoreThanMaxPerUser. They describe behaviour the contracts do not have: per-order redemption tracking, a per-investor cap and non-transferable tokens. A repeated redemption, for example, reverts with ZeroAmount rather than OrderAlreadyRedeemed.

Recommendation

Keep the interfaces in step with the implementation, and remove declarations for behaviour that is not implemented.

> **Tradevu Team's Comment**
>
> *Fixed.*

Informational Severity Issues

I-03 depositUSDC reports misleading errors

Resolved

Path

`TradevuEscrow.sol`

Description

Two cases:

Unknown offering. For an offeringId that does not exist, depositUSDC calls getAmountsAndDate() on the zero address before its zero-address check, so it reverts without a reason instead of with UnsupportedToken. Every other entry point performs the check first.
Nothing minted. For an offering where orders were placed but none was resolved, the required deposit is 0. depositUSDC rejects 0 with ZeroAmount and any other amount with "Insufficient deposit", so the offering can never be funded or closed, and withdrawRemainingUSDC reverts with DepositNotDone. Pending orders can still be refunded by the issuer.

Recommendation

Validate the offering before making any external call, and handle offerings with nothing minted explicitly so they can be closed cleanly.

> **Tradevu Team's Comment**
> *Fixed.*

Informational Severity Issues

I-04 README says the whole balance is burned on redemption

Resolved

Path

`TradevuEscrow`

Description

The README says that on redemption "their whole balance is burned". When the pool cannot cover a holder's full payout, the code burns only the portion that was paid and leaves the rest in the holder's wallet (see issue 6). Those leftover tokens cannot be redeemed for anything, because the pool is empty.

Recommendation

Align the documentation with the partial-burn behaviour, or change the behaviour to match the documentation.

> **Tradevu Team's Comment**
>
> *Fixed*

Informational Severity Issues

I-05 Events misreport amounts and parties

Resolved

Path

`TradevuEscrow.sol, ITradevuEscrow.sol`

Description

OrderPlaced and OrderRejected report the principal only, while the investor pays, or is refunded, the principal plus the investment fee.
OrderResolved uses the same amount field for the number of bill tokens minted, while its sibling events use USDC.
The investment fee sent to adminWallet in resolveOrder emits nothing; InvestmentFeeDeducted is declared but never emitted.
DistributionFeeDeducted is emitted only when the fee is non-zero, while OrderRedeemed is emitted even when nothing is paid.
USDCDeposited always records the escrow itself as receiver, and the depositing issuer is not recorded anywhere.
actionID and offeringId are not indexed, so logs cannot be filtered by order or offering.

Recommendation

Have events report the full amounts actually moved, in consistent units, emit an event for every fee transfer, and record the parties that actually act.

> **Tradevu Team's Comment**
>
> *Fixed.*

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
USDC (IERC20 public usdc)	Token	placeOrder pulls amount + investmentFee via safeTransferFrom; resolveOrder, rejectOrder, redeemBillTokens, depositUSDC and withdrawRemainingUSDC pay out via safeTransfer/safeTransferFrom; unaccountedBalance reads balanceOf; depositUSDC checks allowance	Standard ERC-20 whose base units match the configured bill prices and limits; no fee-on-transfer, no rebasing, no transfer hooks, no blacklist/pause of the escrow, adminWallet, issuer, or batch recipients; the USDC contract itself is honest and not upgraded maliciously	Fee-on-transfer or rebasing makes the booked orderPool, settlementPool and totalEscrowed exceed the actual balance; blacklist/pause of one recipient reverts an entire atomic batch; updateUSDC re-points the asset without migration; USDC decimals are never read on-chain, so a decimals mismatch misprices every offering; USDC issuer can freeze funds

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
@openzeppelin/contracts v5.4.0 - SafeERC20, IERC20, Math	Library	safeTransfer/safeTransferFrom at every USDC movement; Math.mulDiv with Floor for holder shares and Ceil for the required deposit	Library correctly reverts on failed/low-level-returning transfers; mulDiv rounding is exact and overflow-safe	A library defect propagates to every asset movement; SafeERC20 cannot detect fee-on-transfer because it checks the call result, not the balance delta; the fee-cancellation argument depends on the floor/ceil directions remaining paired
@openzeppelin/contracts-upgradeable v5.4.0 - Initializable, AccessControlUpgradeable, ReentrancyGuardUpgradeable, ERC20Upgradeable	Library	One-time initialization (_disableInitializers on logic, initializer on proxies); role checks and role administration; nonReentrant on every asset-moving escrow entry point (placeOrder, resolveOrder, rejectOrder, redeemBillTokens, depositUSDC, withdrawRemainingUSDC); ERC-20 ledger for bill tokens	Library semantics are correct; storage layout is preserved across logic upgrades; DEFAULT_ADMIN_ROLE is the admin of every role unless changed; renounceRole only affects the caller's own membership	Role admin compromise grants or revokes any role; renounceRole can leave a role unstaffed; an upgrade that changes storage layout corrupts role and bill storage; library defect affects all three contracts

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
TradevuFactory (behind FactoryProxy)	External protocol	placeOrder gates on isTokenDeployed(token); redeemBillTokens, depositUSDC, withdrawRemainingUSDC, getNewTrueBillPrice, getUserRedemptionAmount and getClaimEndTime resolve offeringId through getTokenByOfferingId; deployToken creates and records each token	Factory logic and its proxy owner are honest; offeringIdToToken is accurate; only genuine, intended TokenProxies are marked tokenDeployedByFactory; the factory address in the escrow is permanent	A factory upgrade can allowlist an arbitrary token or remap an offeringId, redirecting escrow flows; isTokenDeployed is a single boolean, not a code/role/initialization check; factory maintenance freezes every registry read and therefore several escrow flows; the escrow has no setter for factory, so a dead factory cannot be re-pointed without an escrow logic upgrade
ImplementationAuthority (Ownable)	External protocol	Every TokenProxy stores an authority in its unstructured slot; TokenProxy.implementation() calls getImplementation() on it; the authority owner can updateImplementation	Owner is honest and competent; every replacement implementation is storage-compatible and semantically a Token; the pointer is changed only deliberately	One updateImplementation call re-points every token that uses the authority at once; a malicious or broken implementation can mint, burn, reprice, or freeze all offerings; only a non-zero check is enforced; there is no timelock; owner-key compromise is fleet-wide

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Token implementation (shared logic behind TokenProxy)	External protocol	Escrow calls getAmountsAndDate, getBillDetails, decimals, totalSupply, balanceOf, mint, burn and hasRole on token addresses; holders call ERC-20 functions	The token points at the intended, compatible Token logic; billDetails is immutable after initialize; roles are granted to the intended escrow, issuer, and admin	A logic upgrade can change terms, roles, decimals, or totalSupply mid-lifecycle while the escrow re-reads them; a token initialized with a non-canonical escrow or issuer can be allowlisted but cannot be settled by the intended escrow/issuer; a token bug affects every offering that shares the implementation
EscrowProxy, FactoryProxy, TokenProxy	Privileged role	delegatecall to the current implementation; unstructured slots hold the implementation or authority, owner, and maintenance flag; setMaintenance gates delegated calls; transferProxyOwnership and upgradeTo move control	Each proxyOwner is a controlled protocol address; upgrades are audited and storage-compatible; owners are non-zero	A proxy owner can replace logic and drain or freeze the contract; EscrowProxy/FactoryPro _upgradeTo only rejects the same address, so zero or code-less implementations pass; TokenProxy validates that the new authority resolves to a non-zero implementation; maintenance blocks delegated calls for non-owners; ownership transfer is single-step; a zero owner permanently removes the repair path

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
adminWallet	Privileged role	Receives the investment fee in resolveOrder, the distribution fee in depositUSDC, and the swept remainder in withdrawRemainingUSDC; holds the role that controls updateUSDC	Address is controlled, valid, non-blacklisted, and correct at initialization; the same address is trusted to administer the settlement asset	USDC blacklist/pause of adminWallet reverts fee transfers when a fee is due and therefore blocks fee-bearing resolution and funding; a lost key sends fees and sweep proceeds to an inaccessible address; there is no setter, so rotation requires an escrow logic upgrade; one address serves every offering; the fee recipient is also the updateUSDC admin

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Issuer (ISSUE_ROLE holder on a token)	Token	Calls resolveOrder (receives principal), rejectOrder, depositUSDC (funds redemption), and withdrawRemainingUSDC (sweeps remainder); controls order settlement timing	Issuer will settle orders, has the capital to fund redemption at maturity, does not abandon the offering, and handles actionIDs honestly	Resolved principal is unsecured credit: the issuer is paid before redemption is funded, and depositUSDC has no deadline; unresolved orders can remain indefinitely because there is no investor cancel; rejection and resolution are issuer-discretionary; the issuer can renounce or be revoked, leaving refunds and settlement unstaffable; a blacklisted recipient reverts a whole batch

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Bill-token holders and investors	External protocol	Call placeOrder, ERC-20 transfer/transferFrom/approve, and redeemBillTokens; redemption is based on the caller's current token balance, not on an order	No trust is placed in holders; they may act adversarially within the rules	Any caller can consume an unused actionID; redemption rights move with the token and there is no snapshot; redemptions are first-come-first-served against one finite pool; there is no investor cancel; a blacklisted holder can revert a refund batch that includes them (resolution pays the issuer and admin, not the holder); contract holders can attempt reentrancy (guarded by nonReentrant)

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Factory-deployed bill tokens (TokenProxy → Token)	Token	The token ledger records claims; the escrow is the expected CONTROLLER_ROLE and mints/burns through it; ISSUE_ROLE is checked by the escrow on the token	Each token is initialized with the canonical escrow as CONTROLLER_ROLE, the intended issuer as ISSUE_ROLE, and consistent decimals/terms; the per-token proxy owner is trusted	deployToken accepts arbitrary initialization data and an arbitrary owner_; a payload that succeeds without calling initialize can register an uninitialized token; roles can be re-granted by the token admin; the token is freely transferable with no allowlist; a per-token proxy owner can re-point the token to a different authority
Off-chain actionID and offeringId namespaces; deployment wiring	Other	actionID is the global key for orders; offeringId is the factory registry key and the escrow's claim-state key; admin, issuer, proxyOwner, usdc, and factory addresses are supplied at deployment/initialization	actionIDs are unique and generated for one intended token/user; offeringIds are unique; proxy upgrade and initialize are atomic; every supplied address is correct	actionID is caller-chosen and globally scoped, so any caller can consume a victim's intended ID; there is no on-chain binding to token or user; an unknown offeringId resolves to the zero token; a non-atomic upgrade-plus-initialize permits initialization front-running; a wrong admin/issuer/proxyOwn grants permanent control

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Block timestamp and Base Layer 2 sequencing	Other	Every offering window and the claim window compare block.timestamp to stored dates	Timestamps are monotonic and reasonably accurate; the sequencer includes and orders transactions according to its policy	Deadline transactions can be delayed or ordered around; the claim window is wall-clock and cannot be paused or extended; claimWindow has no upper bound and block.timestamp + claimWindow can overflow, making depositUSDC revert; a missed window is permanent
Role identifier keccak256("escrow.issuer.tradevu")	Other	Token.ISSUE_ROLE and four escrow hasRole checks use the same literal string; the role is granted to the issuer at Token.initialize	Every copy of the literal remains identical across code and upgrades	A one-copy divergence in a future implementation silently breaks issuer settlement or re-scopes the role; there is no compiler-enforced linkage between the token constant and the escrow literals

2. Entry & Exit Points Analysis (Function-Level)

This is the **core of the threat model**. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets **one table**.

Contract Name	Function	Category
TradevuEscrow.sol	`constructor()`	**Main invariant(s)** The logic contract itself can never be initialized; only a proxy can. **Access level** Deployment **What this function can do** Call _disableInitializers() on the implementation contract. **What this function cannot/should not do** Initialize a proxy, set any escrow state, or be called after deployment.
TradevuEscrow.sol	`initialize(address usdc_, address admin, address factory_)`	**Main invariant(s)** Exactly one initialization; all three addresses non-zero; adminWallet and factory are permanent; admin is the sole initial escrow admin. **Access level** Deployment (External initializer; first caller on an uninitialized proxy.) **What this function can do** Set usdc, adminWallet and factory; grant DEFAULT_ADMIN_ROLE to admin; initialize ReentrancyGuard and AccessControl; run once. **What this function cannot/should not do** Be called twice, accept a zero address, change adminWallet or factory later, migrate existing balances, or verify that usdc_/factory_ are contracts with the expected interfaces.

Contract Name	Function	Category
TradevuEscrow .sol	`placeOrder(address token, uint256 amount, string calldata actionID)`	**Main invariant(s)** An actionID is used at most once globally; the full quoted amount is booked to orderPool[token] and totalEscrowed; the cap is latched once and never recomputed; a valid order either exists or the transaction reverts; a positive order cannot mint zero tokens. **Access level** Public (Public, nonReentrant.) **What this function can do** Accept amount + floor(amount * investmentFee / 10000) USDC from the caller; record Order{token, user: msg.sender, amount, resolved: false} under actionID; mark actionIdUsed[actionID]; lazily latch the supply cap discountedBillPrice * 10decimals / issuePrice; enforce minInvestAmount <= amount <= maxInvestment, startDate <= block.timestamp <= maturityDate, tokensToMint != 0, and currentTotalSupply + tokensToMint <= maxTotalSupply; book orderPool[token] and totalEscrowed; pull USDC. ** **What this function cannot/should not do** Mint tokens, refund or cancel an order, accept a zero amount, an empty or already-used actionID, a token not recorded by the factory, an order before startDate or after maturityDate, an order below minInvestAmount or above maxInvestment, an order that rounds to zero tokens, or an order that exceeds the cap; enforce a per-investor aggregate cap (maxInvestment is per order and MoreThanMaxPerUser is unused); reserve cap for pending orders (only minted totalSupply counts); prevent a third party from consuming an unused actionID; measure the USDC actually received; guarantee the issuer will ever resolve or reject.

Contract Name	Function	Category
TradevuEscrow .sol	`resolveOrder(string[] calldata actionIds)`	**Main invariant(s)** Each order is resolved at most once; principal and fee are released exactly once; the minted amount uses the current issuePrice and decimals; minted supply after the call does not exceed the latched cap; the caller is an ISSUE_ROLE holder on that token at payment time. **Access level** Public (External, nonReentrant; per-order ISSUE_ROLE check on order.token.) **What this function can do** For up to MAX_BATCH_SIZE (100) orders: verify the caller holds ISSUE_ROLE on order.token, the order is unresolved, block.timestamp <= maturityDate, and the offering has not been funded; mark it resolved; debit orderPool[token] and totalEscrowed by orderAmount + feeAmount; transfer the investment fee to adminWallet and the principal to msg.sender; mint floor(orderAmount * 10decimals / issuePrice) bill tokens to order.user. ** **What this function cannot/should not do** Resolve after maturity, after depositUSDC, twice, an unknown order, or without ISSUE_ROLE; pay a stored issuer (it pays the current caller); make a batch partially succeed; prevent a single failed transfer or mint from reverting the whole batch; enforce a per-investor cap; make token delivery independent of mint success (a mint revert reverts the whole batch).

Contract Name	Function	Category
TradevuEscrow .sol	`rejectOrder(string[] calldata actionIds)`	**Main invariant(s)** Each order is rejected or resolved at most once; the refund equals the order amount plus the investment fee at rejection time; orderPool(token) is debited exactly; no fee is retained on rejection. **Access level** Public (External, nonReentrant; per-order ISSUE_ROLE check on order.token.) **What this function can do** For up to 100 orders: verify the caller holds ISSUE_ROLE on order.token, the order is unresolved, and block.timestamp >= startDate; mark it resolved; debit orderPool(token) and totalEscrowed by orderAmount + feeAmount; refund that full amount to order.user. **What this function cannot/should not do** Reject an already-resolved order, an unknown order, before startDate, without ISSUE_ROLE, or after the order was resolved; make a batch partially succeed; refund a different amount than the current amount + investmentFee; force the issuer to reject (there is no investor-initiated cancel).

Contract Name	Function	Category
TradevuEscrow.sol	`redeemBillTokens( string calldata offeringId)`	**Main invariant(s)** payout <= settlementPool[token]; settlementPool and totalEscrowed are debited by the payout; tokens are burned in proportion to the amount actually paid, except the zero-share path; a token unit can be paid out at most once: the paid portion is burned, and an unpaid remainder cannot be paid once the pool reaches zero. **Access level** Public (Public, nonReentrant; any current holder.) **What this function can do** Resolve the token from the factory; require claimStarted, block.timestamp <= claimEndTime, and not isWithdrawn[offeringId]; read the caller's full token balance; compute userShare = floor(newTrueBill * userBal / tokenTotalSupply); deduct floor(userShare * distributionFee / 10000); cap the payout at settlementPool[token]; burn the caller's full balance, or the proportional portion when the pool is short (zero when the pool is empty); debit settlementPool and totalEscrowed; update userRedeemedAmount and tokenDetails.claimedAmount; transfer the net USDC if non-zero. **What this function cannot/should not do** Redeem before funding, after claimEndTime, after a sweep, with a zero balance, or on behalf of another holder; pay more than the remaining pool; use the live totalSupply as the denominator (it uses the latched cap); guarantee every holder a full pro-rata share when the pool is dust-short; avoid burning the entire balance when the computed gross share rounds to zero while the pool is non-zero.

Contract Name	Function	Category
TradevuEscrow .sol	`depositUSDC(string calldata offeringId, uint256 amount)`	**Main invariant(s)** One funding per offeringId; the exact required amount is pulled; settlementPool is the post-fee deposit; claimEndTime is fixed at funding; depositedAmount is the gross amount; the claim window opens in the funding block. **Access level** Public (External, nonReentrant; ISSUE_ROLE on the token.) **What this function can do** After maturityDate, once per offeringId, and only for an ISSUE_ROLE holder: resolve the token; require amount == ceil(trueBillPrice * totalSupplyMinted / tokenTotalSupply) exactly; require allowance >= amount; pull the USDC; pay floor(amount * distributionFee / 10000) to adminWallet; set depositedAmount[token] = amount, settlementPool[token] = amount - fee, totalEscrowed += amount - fee, and tokenDetails (newTrueBill, feeDeducted, claimStarted, claimEndTime = block.timestamp + claimWindow). **What this function cannot/should not do** Deposit at or before maturity, twice for the same offeringId, zero, without ISSUE_ROLE, with an underpayment or overpayment (exact equality is required), or when no orders were placed (tokenTotalSupply == 0); migrate an old USDC balance; avoid paying the distribution fee; guarantee the issuer will ever call it.

Contract Name	Function	Category
TradevuEscrow.sol	`withdrawRemainingUSDC(string calldata offeringId)`	**Main invariant(s)** The sweep happens only after every redemption right has expired; isWithdrawn is true only after a successful sweep; settlementPool is zero after the call; totalEscrowed is debited by the swept amount. **Access level** Public (External, nonReentrant; ISSUE_ROLE on the token.) **What this function can do** After claimEndTime, once, for an ISSUE_ROLE holder: require the offering was funded and not already withdrawn; read settlementPool[token]; mark isWithdrawn[offeringId] = true; zero the pool; debit totalEscrowed; transfer the remainder to adminWallet. **What this function cannot/should not do** Sweep before funding, before claimEndTime, twice, without ISSUE_ROLE, or with a zero remainder (NoUSDCLeft reverts); send the remainder to the issuer; recover the remainder after a successful sweep; mark a zero-remainder offering as settled (the revert undoes the state writes).
TradevuEscrow.sol	`updateUSDC(address usdc_)`	**Main invariant(s)** Only DEFAULT_ADMIN_ROLE; the new address is non-zero; existing ledger entries keep their old-asset units. **Access level** Owner (DEFAULT_ADMIN_ROLE on the escrow.) **What this function can do** Replace the settlement asset with any non-zero address; every later transfer and balanceOf uses the new token. **What this function cannot/should not do** Migrate the old balance, adjust totalEscrowed or any pool, check the new token's code or decimals, emit an event (no event is declared), be called by a non-admin, or be undone without a second call.

Contract Name	Function	Category
TradevuEscrow .sol	`unaccountedBalance()`	**Main invariant(s)** The global balance is at least totalEscrowed; otherwise the subtraction reverts. A healthy escrow returns zero or a small rounding surplus. **Access level** Public **What this function can do** Return usdc.balanceOf(address(this)) - totalEscrowed. **What this function cannot/should not do** Change state; prove per-offering solvency; account for assets left in an old USDC after updateUSDC; account for ETH; distinguish which offering holds the surplus.
TradevuEscrow .sol	`getOrder(string calldata actionId)`	**Main invariant(s)** The tuple matches the Order struct; an unknown ID is all zeros. **Access level** Public **What this function can do** Return (token, user, amount, resolved) for an actionID. **What this function cannot/should not do** Return a redeemed flag, an offeringId, or a fee/price snapshot; verify that the order exists (an unknown ID returns zeros); return the ITradevuEscrow.OrderDetails shape, which lists user first and includes orderRedeemed.

Contract Name	Function	Category
TradevuEscrow.sol	`getUserRedeemedAmount(address token, address user)`	**Main invariant(s)** The value is monotonic for a user/token and increases only by the net USDC actually transferred. **Access level** Public **What this function can do** Return the cumulative USDC paid to user for token across successful redemptions. **What this function cannot/should not do** Return a token amount despite the name/comment; separate principal from fee; include an unpaid remainder left after a pool-capped redemption.
TradevuEscrow.sol	`getTokenTotalSupply(address token)`	**Main invariant(s)** The cap is fixed after the first placeOrder and is the redemption denominator. **Access level** Public **What this function can do** Return the latched supply cap for an offering. **What this function cannot/should not do** Recompute the cap from current terms; change it; return a value before the first order (it returns zero).

Contract Name	Function	Category
TradevuEscrow.sol	`getNewTrueBillPrice(string calldata offeringId)`	**Main invariant(s)** Before funding the quote mirrors depositUSDC's formula; after funding it mirrors depositedAmount. **Access level** Public (Public view; reverts on unknown/uncapped offerings.) **What this function can do** Quote the exact USDC depositUSDC will require: before funding, ceil(trueBillPrice * totalSupplyMinted / tokenTotalSupply); after funding, the stored depositedAmount(token). **What this function cannot/should not do** Quote before a cap exists (NoOrdersPlaced), resolve an unknown offeringId (UnsupportedToken), use live supply after funding (it returns the stored amount), migrate assets, or include the distribution fee.
TradevuEscrow.sol	`getIsDeposited(string calldata offeringId)`	**Main invariant(s)** True only after a successful depositUSDC for that exact offeringId. **Access level** Public **What this function can do** Return isDeposited(offeringId). **What this function cannot/should not do** Verify that the mapped token is valid or that the claim window is open.

Contract Name	Function	Category
TradevuEscrow .sol	`getUserRedemptionAmount( string calldata offeringId, address user)`	**Main invariant(s)** The quote is the same formula as the uncapped part of redeemBillTokens; it can exceed the actual payout when the pool is short or the window is closed. **Access level** Public **What this function can do** Quote the holder's net share using tokenDetails[token].newTrueBill, the latched cap, and the current distributionFee. **What this function cannot/should not do** Apply the settlementPool cap; check claimStarted, claimEndTime, or isWithdrawn; guarantee the quoted amount will be paid; account for the proportional-burn path; return a value for an unknown offering.
TradevuEscrow .sol	`getClaimEndTime(string calldata offeringId)`	**Main invariant(s)** Zero before funding; depositUSDC timestamp plus claimWindow after funding. **Access level** Public **What this function can do** Return tokenDetails[token].claimEndTime. **What this function cannot/should not do** Derive the time from maturity, verify that funding occurred, or extend the window.

Contract Name	Function	Category
TradevuEscrow.sol	`usdc()`	**Main invariant(s)** Returns the current settlement asset. **Access level** Public **What this function can do** Read the configured USDC address. **What this function cannot/should not do** Change it except through updateUSDC or an escrow logic upgrade.
TradevuEscrow.sol	`factory()`	**Main invariant(s)** Returns the configured factory. **Access level** Public **What this function can do** Read the permanent factory address. **What this function cannot/should not do** Change it without an escrow logic upgrade.
TradevuEscrow.sol	`adminWallet()`	**Main invariant(s)** Returns the configured payee. **Access level** Public **What this function can do** Read the permanent fee and sweep recipient. **What this function cannot/should not do** Change it without an escrow logic upgrade.

Contract Name	Function	Category
TradevuEscrow .sol	`actionIdUsed(string)`	**Main invariant(s)** True after a successful placeOrder for that ID. **Access level** Public **What this function can do** Read whether an actionID has been consumed. **What this function cannot/should not do** Write the flag.
TradevuEscrow .sol	`orders(string)`	**Main invariant(s)** An unknown ID returns zeros. **Access level** Public **What this function can do** Read (token, user, amount, resolved) for an actionID. **What this function cannot/should not do** Return a redeemed flag or write the order.
TradevuEscrow .sol	`tokenDetails(address)`	**Main invariant(s)** Reflects the funded/redemption state for the token. **Access level** Public **What this function can do** Read (feeDeducted, claimStarted, newTrueBill, claimEndTime, claimedAmount). **What this function cannot/should not do** Write the struct.

Contract Name	Function	Category
TradevuEscrow.sol	`tokenTotalSupply(address)`	**Main invariant(s)** Zero before the first order; fixed afterwards. **Access level** Public **What this function can do** Read the latched supply cap. **What this function cannot/should not do** Recompute or write the cap.
TradevuEscrow.sol	`isDeposited(string)`	**Main invariant(s)** True after a successful depositUSDC for that ID. **Access level** Public **What this function can do** Read the funding flag for an offeringId. **What this function cannot/should not do** Write the flag.
TradevuEscrow.sol	`settlementPool(address)`	**Main invariant(s)** Debited by redemptions and the final sweep. **Access level** Public **What this function can do** Read the per-token redemption pool. **What this function cannot/should not do** Write the pool.

Contract Name	Function	Category
TradevuEscrow.sol	`isWithdrawn(string)`	**Main invariant(s)** True only after a successful sweep. **Access level** Public **What this function can do** Read the sweep flag for an offeringId. **What this function cannot/should not do** Write the flag.
TradevuEscrow.sol	`depositedAmount(address)`	**Main invariant(s)** Fixed at funding time. **Access level** Public **What this function can do** Read the gross deposit for a token. **What this function cannot/should not do** Write the amount.
TradevuEscrow.sol	`orderPool(address)`	**Main invariant(s)** Debited by the settled order's amount on resolve or reject. **Access level** Public **What this function can do** Read the per-token pending-order pool. **What this function cannot/should not do** Write the pool.

Contract Name	Function	Category
TradevuEscrow.sol	`totalEscrowed()`	**Main invariant(s)** Equals the sum of all orderPool and settlementPool entries. **Access level** Public **What this function can do** Read the escrow's total obligations. **What this function cannot/should not do** Write the total.
TradevuEscrow.sol	`MAX_BATCH_SIZE()`	**Main invariant(s)** The value is 100. **Access level** Public **What this function can do** Read the batch limit constant. **What this function cannot/should not do** Change the constant.
TradevuEscrow.sol	`DEFAULT_ADMIN_ROLE()`	**Main invariant(s)** The id is 0x00. **Access level** Public **What this function can do** Read the default admin role id (0x00). **What this function cannot/should not do** Change the constant.

Contract Name	Function	Category
TradevuEscrow .sol	`getRoleAdmin(bytes32)`	**Main invariant(s)** Defaults to DEFAULT_ADMIN_ROLE unless changed. **Access level** Public **What this function can do** Read the admin role for a role. **What this function cannot/should not do** Change the admin role.
TradevuEscrow .sol	`grantRole(bytes32, address)`	**Main invariant(s)** Only the role admin can grant. **Access level** Owner **What this function can do** The role admin can grant a role. **What this function cannot/should not do** Be called by a non-admin or bypass the role admin.
TradevuEscrow .sol	`revokeRole(bytes32, address)`	**Main invariant(s)** Only the role admin can revoke. **Access level** Owner **What this function can do** The role admin can revoke a role. **What this function cannot/should not do** Be called by a non-admin or bypass the role admin.

Contract Name	Function	Category
TradevuEscrow.sol	`renounceRole(bytes32, address)`	**Main invariant(s)** Only the caller's own membership can be renounced. **Access level** Role holder (Role holder, self only.) **What this function can do** A role holder can renounce its own membership. **What this function cannot/should not do** Renounce another account's role.
TradevuEscrow.sol	`hasRole(bytes32, address)`	**Main invariant(s)** Reflects the current role state. **Access level** Public **What this function can do** Read whether an account holds a role. **What this function cannot/should not do** Change membership.
TradevuEscrow.sol	`supportsInterface(bytes4)`	**Main invariant(s)** Library-defined interface support. **Access level** Public **What this function can do** Read ERC-165 interface support. **What this function cannot/should not do** Change the result.

Contract Name	Function	Category
Token.sol	`constructor()`	**Main invariant(s)** The implementation cannot be initialized directly. **Access level** Deployment **What this function can do** Call _disableInitializers() on the shared implementation. **What this function cannot/should not do** Initialize a proxy or set token state.

Contract Name	Function	Category
Token.sol	`initialize(string name, string symbol, uint8 decimals_, address escrow, address issuer, address admin, BillDetails billDetails_)`	**Main invariant(s)** One initialization; terms and decimals are fixed at initialization; roles are fixed at initialization but mutable afterwards by DEFAULT_ADMIN_ROLE; startDate is captured against the initialization block. **Access level** Deployment (External initializer; first caller on the proxy; the factory calls it atomically through the proxy fallback in deployToken.) **What this function can do** Set ERC-20 metadata and _decimals; grant DEFAULT_ADMIN_ROLE to admin, CONTROLLER_ROLE to escrow, and ISSUE_ROLE to issuer; store the ten bill fields; require non-zero escrow/issuer/admin, decimals_ <= 18, each fee <= 10000, issuePrice != 0, discountedBillPrice != 0, discountedBillPrice <= trueBillPrice, maxInvestment != 0, minInvestAmount <= maxInvestment, claimWindow != 0, startDate >= block.timestamp, and maturityDate > startDate; run once. **What this function cannot/should not do** Be called twice; accept a zero role address; change any term later (no setters); enforce an upper bound on claimWindow; enforce a relationship between issuePrice and the investment band; enforce a bound on investmentFee + distributionFee; verify that escrow is the canonical escrow or that issuer is solvent; prevent the first caller from initializing a directly deployed, uninitialized TokenProxy.

Contract Name	Function	Category
Token.sol	`mint(address user, uint256 amount)`	**Main invariant(s)** Only CONTROLLER_ROLE mints; the token-level window matches the offering window; totalSupply increases by exactly amount. **Access level** Role holder (CONTROLLER_ROLE; the escrow is the expected holder.) **What this function can do** Mint amount to user while startDate <= block.timestamp <= maturityDate, for a non-zero amount and non-zero user, only for CONTROLLER_ROLE. **What this function cannot/should not do** Mint before the start or after maturity; mint zero; enforce the offering's supply cap (the escrow does that); be called by a non-controller; burn; prevent the controller from minting to any address.
Token.sol	`burn(address user, uint256 amount)`	**Main invariant(s)** Only CONTROLLER_ROLE burns; burns are only at/after maturity; the user's balance decreases by amount. **Access level** Role holder (CONTROLLER_ROLE; the escrow is the expected holder.) **What this function can do** Burn amount from any user at or after maturityDate, only for CONTROLLER_ROLE, with no allowance and no per-holder consent. **What this function cannot/should not do** Burn before maturity; burn more than the user's balance (the ERC-20 balance check reverts); be called by a non-controller; restrict which holder is burned.

Contract Name	Function	Category
Token.sol	`decimals()`	**Main invariant(s)** decimals() == _decimals set at initialization. **Access level** Public **What this function can do** Return _decimals, capped at 18 by initialize. **What this function cannot/should not do** Change after initialization; be read from the underlying USDC (it is the bill token's own decimals).
Token.sol	`getInvestmentFee()`	**Main invariant(s)** The value is the stored immutable fee (basis points). **Access level** Public **What this function can do** Return billDetails.investmentFee as uint256. **What this function cannot/should not do** Change the fee; validate it against the current escrow expectations.
Token.sol	`getDistributionFee()`	**Main invariant(s)** The value is the stored immutable fee (basis points). **Access level** Public **What this function can do** Return billDetails.distributionFee as uint256. **What this function cannot/should not do** Change the fee; enforce that the fee plus the investment fee is below 10000.

Contract Name	Function	Category
Token.sol	`getAmountsAndDate()`	**Main invariant(s)** The tuple matches the stored bill fields. **Access level** Public **What this function can do** Return (maxInvestment, minInvestAmount, startDate, maturityDate). **What this function cannot/should not do** Change the values; return a different tuple order.
Token.sol	`getBillAmount()`	**Main invariant(s)** The tuple matches the stored bill fields. **Access level** Public **What this function can do** Return (trueBillPrice, discountedBillPrice). **What this function cannot/should not do** Change either price.
Token.sol	`getBillDetails()`	**Main invariant(s)** The returned struct matches storage field-for-field. **Access level** Public **What this function can do** Return the full BillDetails struct. **What this function cannot/should not do** Change any field; validate the fields at read time.

Contract Name	Function	Category
Token.sol	`billDetails()`	**Main invariant(s)** Field order matches the BillDetails struct. **Access level** Public **What this function can do** Read the ten bill fields in declaration order. **What this function cannot/should not do** Write or reorder the fields.
Token.sol	`CONTROLLER_ROLE()`	**Main invariant(s)** The id is constant. **Access level** Public **What this function can do** Read keccak256("tradevu.controller.role"). **What this function cannot/should not do** Change the role id.
Token.sol	`ISSUE_ROLE()`	**Main invariant(s)** The id must match the escrow's hardcoded literal. **Access level** Public **What this function can do** Read keccak256("escrow.issuer.tradevu"). **What this function cannot/should not do** Change the role id.

Contract Name	Function	Category
Token.sol	`transfer(address, uint256)`	**Main invariant(s)** Standard ERC-20 accounting; the redemption claim moves with the token. **Access level** Public **What this function can do** Move bill tokens to another address. **What this function cannot/should not do** Restrict transfers or preserve the seller's redemption right.
Token.sol	`transferFrom(address, address,uint256)`	**Main invariant(s)** Standard ERC-20 allowance accounting. **Access level** Public **What this function can do** Move bill tokens using an allowance. **What this function cannot/should not do** Bypass the allowance.
Token.sol	`approve(address, uint256)`	**Main invariant(s)** Standard ERC-20 allowance accounting. **Access level** Public **What this function can do** Set an allowance for a spender. **What this function cannot/should not do** Force a holder to approve.

Contract Name	Function	Category
Token.sol	`allowance(address, address)`	**Main invariant(s)** Standard ERC-20 allowance accounting. **Access level** Public **What this function can do** Read an allowance. **What this function cannot/should not do** Change the allowance.
Token.sol	`balanceOf(address)`	**Main invariant(s)** Redemption uses this live balance against the latched cap. **Access level** Public **What this function can do** Read a holder's bill-token balance. **What this function cannot/should not do** Change the balance.
Token.sol	`totalSupply()`	**Main invariant(s)** Mint and burn are the only supply changes. **Access level** Public **What this function can do** Read the live bill-token supply. **What this function cannot/should not do** Change the supply directly.

Contract Name	Function	Category
Token.sol	`name()`	**Main invariant(s)** Metadata is fixed at initialization. **Access level** Public **What this function can do** Read the token name. **What this function cannot/should not do** Change the name.
Token.sol	`symbol()`	**Main invariant(s)** Metadata is fixed at initialization. **Access level** Public **What this function can do** Read the token symbol. **What this function cannot/should not do** Change the symbol.
Token.sol	`DEFAULT_ADMIN_ROLE()`	**Main invariant(s)** The id is 0x00. **Access level** Public **What this function can do** Read the default admin role id (0x00). **What this function cannot/should not do** Change the constant.

Contract Name	Function	Category
Token.sol	`getRoleAdmin(bytes32)`	**Main invariant(s)** Defaults to DEFAULT_ADMIN_ROLE unless changed. **Access level** Public **What this function can do** Read the admin role for a role. **What this function cannot/should not do** Change the admin role.
Token.sol	`grantRole(bytes32, address)`	**Main invariant(s)** Only the role admin can grant. **Access level** Owner **What this function can do** The role admin can grant a role. **What this function cannot/should not do** Be called by a non-admin or bypass the role admin.
Token.sol	`revokeRole(bytes32, address)`	**Main invariant(s)** Only the role admin can revoke. **Access level** Owner **What this function can do** The role admin can revoke a role. **What this function cannot/should not do** Be called by a non-admin or bypass the role admin.

Contract Name	Function	Category
Token.sol	`renounceRole(bytes32, address)`	**Main invariant(s)** Only the caller's own membership can be renounced. **Access level** Role holder (Role holder, self only.) **What this function can do** A role holder can renounce its own membership. **What this function cannot/should not do** Renounce another account's role.
Token.sol	`hasRole(bytes32, address)`	**Main invariant(s)** Reflects the current role state. **Access level** Public **What this function can do** Read whether an account holds a role. **What this function cannot/should not do** Change membership.
Token.sol	`supportsInterface(bytes4)`	**Main invariant(s)** Library-defined interface support. **Access level** Public **What this function can do** Read ERC-165 interface support. **What this function cannot/should not do** Change the result.

Contract Name	Function	Category
TradevuFactory .sol	`constructor()`	**Main invariant(s)** The implementation cannot be initialized directly. **Access level** Deployment **What this function can do** Call _disableInitializers() on the factory implementation. **What this function cannot/should not do** Initialize the proxy or set factory state.
TradevuFactory .sol	`initialize(address admin_, address impAuth)`	**Main invariant(s)** One initialization; admin_ holds both roles; the authority pointer is fixed until a factory logic upgrade. **Access level** Deployment (External initializer; first caller on the proxy.) **What this function can do** Grant DEFAULT_ADMIN_ROLE and ADMIN_ROLE to admin_; set implementationAuthority = impAuth; run once. **What this function cannot/should not do** Be called twice; change implementationAuthority later (no setter); zero-check impAuth (it is intentionally not checked); grant roles to any other address; validate that impAuth is a contract.

Contract Name	Function	Category
TradevuFactory .sol	`deployToken(string calldata offeringId_, address owner_, bytes memory data)`	**Main invariant(s)** offeringId_ is unique; each call creates a new proxy; the token is recorded exactly once; only ADMIN_ROLE can create offerings; the token becomes escrow-accepted because isTokenDeployed is set. **Access level** Owner **What this function can do** For ADMIN_ROLE: require a non-empty, unused offeringId_; deploy new TokenProxy(implementationAuthority, owner_); mark tokenDeployedByFactory[proxy], offeringIdUsed[offeringId_], offeringIdToToken[offeringId_]; push to tokens; staticcall implementation() and require the decoded address to be non-zero; execute the arbitrary data payload with a low-level call; require the call to succeed; read getInvestmentFee() and getDistributionFee() from the token; emit TokenDeployed. **What this function cannot/should not do** Verify that data is Token.initialize; verify the initialization outcome beyond call success; verify that owner_ is non-zero or trusted; verify the token's roles, escrow, issuer, decimals, or terms; verify that the reported fees are non-zero; revoke or upgrade the token; initialize the token itself; prevent a payload that calls a proxy view (for example implementation()) from succeeding without initializing.

Contract Name	Function	Category
TradevuFactory .sol	`isTokenDeployed( address token)`	**Main invariant(s)** True exactly for addresses recorded by deployToken (unless the factory logic is upgraded). **Access level** Public **What this function can do** Return tokenDeployedByFactory(token). **What this function cannot/should not do** Verify that the address has code, is initialized, has the expected roles, or still points at the intended logic.
TradevuFactory .sol	`getTokenByOfferingId( string calldata offeringId_)`	**Main invariant(s)** An offeringId maps to at most one token; an unused ID returns the zero address. **Access level** Public **What this function can do** Return offeringIdToToken(offeringId_). **What this function cannot/should not do** Verify that the mapped token exists or is initialized; return more than one token per ID (the mapping is one-to-one by construction).
TradevuFactory .sol	`isOfferingIdUsed( string calldata offeringId_)`	**Main invariant(s)** True only after a successful deployToken for that ID. **Access level** Public **What this function can do** Return offeringIdUsed(offeringId_). **What this function cannot/should not do** Enumerate offerings or prove that the mapped token is valid.

Contract Name	Function	Category
TradevuFactory .sol	`implementationAuthority()`	**Main invariant(s)** Returns the configured authority. **Access level** Public **What this function can do** Read the factory's authority pointer. **What this function cannot/should not do** Change it without a factory logic upgrade.
TradevuFactory .sol	`offeringIdUsed(string)`	**Main invariant(s)** True after a successful deployToken. **Access level** Public **What this function can do** Read whether an offeringId has been used. **What this function cannot/should not do** Write the flag.
TradevuFactory .sol	`offeringIdToToken( string)`	**Main invariant(s)** An unused ID returns the zero address. **Access level** Public **What this function can do** Read the token mapped to an offeringId. **What this function cannot/should not do** Write the mapping.

Contract Name	Function	Category
TradevuFactory .sol	`tokens(uint256)`	**Main invariant(s)** Out-of-bounds indices revert. **Access level** Public **What this function can do** Read the token array by index. **What this function cannot/should not do** Write the array or return its length (no length getter).
TradevuFactory .sol	`ADMIN_ROLE()`	**Main invariant(s)** The id is constant. **Access level** Public **What this function can do** Read keccak256("tradevu.factory.admin"). **What this function cannot/should not do** Change the role id.
TradevuFactory .sol	`DEFAULT_ADMIN_ROLE()`	**Main invariant(s)** The id is 0x00. **Access level** Public **What this function can do** Read the default admin role id (0x00). **What this function cannot/should not do** Change the constant.

Contract Name	Function	Category
TradevuFactory .sol	`getRoleAdmin(bytes32)`	**Main invariant(s)** Defaults to DEFAULT_ADMIN_ROLE unless changed. **Access level** Public **What this function can do** Read the admin role for a role. **What this function cannot/should not do** Change the admin role.
TradevuFactory .sol	`grantRole(bytes32, address)`	**Main invariant(s)** Only the role admin can grant. **Access level** Owner **What this function can do** The role admin can grant a role. **What this function cannot/should not do** Be called by a non-admin or bypass the role admin.
TradevuFactory .sol	`revokeRole(bytes32, address)`	**Main invariant(s)** Only the role admin can revoke. **Access level** Owner **What this function can do** The role admin can revoke a role. **What this function cannot/should not do** Be called by a non-admin or bypass the role admin.

Contract Name	Function	Category
TradevuFactory.sol	`renounceRole(bytes32, address)`	**Main invariant(s)** Only the caller's own membership can be renounced. **Access level** Role holder (Role holder, self only.) **What this function can do** A role holder can renounce its own membership. **What this function cannot/should not do** Renounce another account's role.
TradevuFactory.sol	`hasRole(bytes32, address)`	**Main invariant(s)** Reflects the current role state. **Access level** Public **What this function can do** Read whether an account holds a role. **What this function cannot/should not do** Change membership.
TradevuFactory.sol	`supportsInterface(bytes4)`	**Main invariant(s)** Library-defined interface support. **Access level** Public **What this function can do** Read ERC-165 interface support. **What this function cannot/should not do** Change the result.

Contract Name	Function	Category
Implementation Authority.sol	`constructor(address implementation, address owner_)`	**Main invariant(s)** The initial implementation is non-zero; ownership is set once at deployment. **Access level** Deployment **What this function can do** Require a non-zero implementation; set implementation; initialize Ownable with owner (OpenZeppelin rejects a zero owner); emit UpdatedImplementation. **What this function cannot/should not do** Set a zero implementation; set a zero owner; validate storage compatibility of the implementation; set multiple implementations.
Implementation Authority.sol	`updateImplementation( address _newImplementation)`	**Main invariant(s)** Only the owner; the pointer is non-zero; one pointer serves every dependent token. **Access level** Owner **What this function can do** For the Ownable owner: set _implementation to any non-zero address; emit UpdatedImplementation; affect every TokenProxy that points at this authority. **What this function cannot/should not do** Set zero; validate that the new implementation is a compatible Token; enforce a timelock; affect tokens that have been re-pointed to another authority; be called by a non-owner.

Contract Name	Function	Category
Implementation Authority.sol	`getImplementation()`	**Main invariant(s)** Returns _implementation, which is non-zero by construction/update. **Access level** Public **What this function can do** Return the current implementation address. **What this function cannot/should not do** Change state or validate that the address has code.
Implementation Authority.sol	`owner()`	**Main invariant(s)** There is exactly one owner. **Access level** Public **What this function can do** Read the authority owner. **What this function cannot/should not do** Change the owner.
Implementation Authority.sol	`transferOwnership( address)`	**Main invariant(s)** Only the owner can transfer; zero is rejected. **Access level** Owner **What this function can do** Transfer ownership in one step. **What this function cannot/should not do** Use a two-step transfer or transfer to zero.

Contract Name	Function	Category
Implementation Authority.sol	`renounceOwnership()`	**Main invariant(s)** Renunciation permanently removes updateImplementation. **Access level** Owner **What this function can do** Renounce ownership, setting the owner to zero. **What this function cannot/should not do** Recover ownership or the update path afterwards.
EscrowProxy .sol	`constructor(address owner_)`	**Main invariant(s)** The owner slot is set; the implementation slot is zero until upgradeTo. **Access level** Deployment **What this function can do** Store owner_ in the unstructured proxyOwner slot. **What this function cannot/should not do** Set an implementation (the proxy starts empty), initialize the escrow, or reject a zero owner.
EscrowProxy .sol	`maintenance()`	**Main invariant(s)** The flag is independent of the implementation slot. **Access level** Public **What this function can do** Read the maintenance flag from its unstructured slot. **What this function cannot/should not do** Change state; indicate whether the implementation is valid.

Contract Name	Function	Category
EscrowProxy .sol	`setMaintenance(bool _maintenance)`	**Main invariant(s)** Only the owner toggles the flag; the flag gates only the fallback path. **Access level** Owner **What this function can do** For the proxy owner: set the maintenance flag; when true, _fallback allows only the proxy owner to use delegated escrow calls. **What this function cannot/should not do** Be called by a non-owner; block the owner's own delegated calls or direct proxy functions; emit an event (none is declared); prevent the owner from upgrading during maintenance.
EscrowProxy .sol	`proxyOwner()`	**Main invariant(s)** Returns the stored owner, possibly zero. **Access level** Public **What this function can do** Read the owner from its unstructured slot. **What this function cannot/should not do** Change the owner; validate that the owner is non-zero.

Contract Name	Function	Category
EscrowProxy .sol	`transferProxyOwnership( address newOwner)`	**Main invariant(s)** The owner slot is replaced atomically; ownership is single-step. **Access level** Owner **What this function can do** For the proxy owner: require newOwner != 0; emit ProxyOwnershipTransferred; set the new owner. **What this function cannot/should not do** Two-step transfer; transfer to zero; be called by a non-owner; prevent a typo from permanently losing the upgrade/maintenance path.
EscrowProxy .sol	`upgradeTo(address newImplementation)`	**Main invariant(s)** The implementation slot is replaced only by a different address. **Access level** Owner **What this function can do** For the proxy owner: require the new address differs from the current slot; store it as the implementation; emit Upgraded. **What this function cannot/should not do** Reject the zero address or a code-less address; validate storage compatibility; use a two-step upgrade; be called by a non-owner; preserve the old implementation for rollback.

Contract Name	Function	Category
EscrowProxy .sol	`upgradeToAndCall( address newImplementation, bytes memory data)`	**Main invariant(s)** Upgrade and initialization call are atomic; the implementation change survives only if the call succeeds. **Access level** Owner **What this function can do** For the proxy owner: set the implementation, then address(this).call{value: msg.value}(data); require success; if the call reverts, the whole transaction reverts and the upgrade is undone. **What this function cannot/should not do** Validate the payload; partially apply the upgrade; execute a delegated payload while maintenance is enabled (the self-call is treated as a non-owner call and reverts, although direct proxy functions remain reachable); recover ETH left in the proxy; be called by a non-owner.
EscrowProxy .sol	`implementation()`	**Main invariant(s)** Returns the stored slot, possibly zero. **Access level** Public **What this function can do** Read the implementation address from its unstructured slot. **What this function cannot/should not do** Validate code, initialize, or resolve through an authority (this proxy stores the implementation directly).

Contract Name	Function	Category
EscrowProxy .sol	`fallback()`	**Main invariant(s)** All escrow logic is reached only through delegatecall; the implementation slot must be non-zero. **Access level** Public (Public; maintenance-restricted for non-owners.) **What this function can do** If maintenance is false, or the caller is the proxy owner, delegatecall the current implementation with the full calldata and return/revert with its returndata. **What this function cannot/should not do** Bypass maintenance for non-owners; handle a zero implementation (it reverts); recover ETH; validate selectors.
EscrowProxy .sol	`receive()`	**Main invariant(s)** No ETH custody path exists in the implementation. **Access level** Public **What this function can do** Invoke the same _fallback path for empty calldata. **What this function cannot/should not do** Accept ETH unless the implementation has a receive/fallback; the current TradevuEscrow has neither, so a plain ETH transfer reverts.

Contract Name	Function	Category
FactoryProxy .sol	`constructor(address owner_)`	**Main invariant(s)** The owner slot is set; the implementation slot is zero until upgradeTo. **Access level** Deployment **What this function can do** Store owner_ in the com.factoryproxy.owner slot. **What this function cannot/should not do** Set an implementation, initialize the factory, or reject a zero owner.
FactoryProxy .sol	`maintenance()`	**Main invariant(s)** The flag is independent of the implementation slot. **Access level** Public **What this function can do** Read the factory proxy maintenance flag. **What this function cannot/should not do** Change state; indicate whether the factory logic is valid.
FactoryProxy .sol	`setMaintenance(bool _maintenance)`	**Main invariant(s)** Only the owner toggles the flag; the flag gates the fallback. **Access level** Owner **What this function can do** For the proxy owner: set the flag; when true, non-owners cannot use delegated factory calls. **What this function cannot/should not do** Be called by a non-owner; emit an event (none is declared); block direct proxy functions or the owner's own calls.

Contract Name	Function	Category
FactoryProxy .sol	`proxyOwner()`	**Main invariant(s)** Returns the stored owner, possibly zero. **Access level** Public **What this function can do** Read the owner. **What this function cannot/should not do** Change the owner or validate that it is non-zero.
FactoryProxy .sol	`transferProxyOwnership( address newOwner)`	**Main invariant(s)** Single-step owner replacement. **Access level** Owner **What this function can do** For the proxy owner: require non-zero; emit ProxyOwnershipTransferred; set the new owner. **What this function cannot/should not do** Two-step transfer; transfer to zero; be called by a non-owner.
FactoryProxy .sol	`upgradeTo(address newImplementation)`	**Main invariant(s)** The implementation slot is replaced only by a different address. **Access level** Owner **What this function can do** For the proxy owner: require the new address differs from the current implementation slot; store it; emit Upgraded. **What this function cannot/should not do** Reject zero or code-less implementations; validate storage compatibility; be called by a non-owner.

Contract Name	Function	Category
FactoryProxy .sol	`upgradeToAndCall( address newImplementation, bytes memory data)`	**Main invariant(s)** Upgrade and initialization call are atomic. **Access level** Owner **What this function can do** For the proxy owner: set the implementation, then self-call data with msg.value; require success; roll back the upgrade on failure. **What this function cannot/should not do** Validate the payload; execute a delegated payload while maintenance is enabled (the self-call is a non-owner call and reverts, although direct proxy functions remain reachable); recover ETH; be called by a non-owner.
FactoryProxy .sol	`implementation()`	**Main invariant(s)** Returns the stored slot, possibly zero. **Access level** Public **What this function can do** Read the implementation address directly from the com.factoryproxy.implementation slot. **What this function cannot/should not do** Validate code or resolve through an authority (the factory stores the implementation directly).

Contract Name	Function	Category
FactoryProxy .sol	`fallback()`	**Main invariant(s)** All factory logic is reached through delegatecall; the implementation slot must be non-zero. **Access level** Public (Public; maintenance-restricted for non-owners.) **What this function can do** Delegatecall the implementation with the full calldata, subject to the maintenance check. **What this function cannot/should not do** Bypass maintenance for non-owners; handle a zero implementation (it reverts); validate selectors; recover ETH.
FactoryProxy .sol	`receive()`	**Main invariant(s)** No ETH custody path exists in the factory implementation. **Access level** Public **What this function can do** Invoke _fallback for empty calldata. **What this function cannot/should not do** Accept plain ETH because TradevuFactory has no receive/fallback.

Contract Name	Function	Category
TokenProxy.sol	`constructor(address impAuth, address owner_)`	**Main invariant(s)** The authority and owner slots are set; the token logic is resolved dynamically. **Access level** Deployment **What this function can do** Store impAuth in the com.proxy.implementationAuthority slot and owner_ in the owner slot. **What this function cannot/should not do** Validate that impAuth is non-zero or a real authority; validate owner_; initialize the token.
TokenProxy.sol	`maintenance()`	**Main invariant(s)** The flag is independent of the authority slot. **Access level** Public **What this function can do** Read the maintenance flag. **What this function cannot/should not do** Change state; resolve the implementation.
TokenProxy.sol	`setMaintenance(bool _maintenance)`	**Main invariant(s)** Only the owner toggles the flag; the flag gates the fallback. **Access level** Owner **What this function can do** For the proxy owner: set the flag; when true, non-owners cannot use delegated token calls. **What this function cannot/should not do** Be called by a non-owner; emit an event; block direct proxy functions or the owner's own calls.

Contract Name	Function	Category
TokenProxy.sol	`proxyOwner()`	**Main invariant(s)** Returns the stored owner, possibly zero. **Access level** Public **What this function can do** Read the owner. **What this function cannot/should not do** Change the owner or validate that it is non-zero.
TokenProxy.sol	`transferProxyOwnership( address newOwner)`	**Main invariant(s)** Single-step owner replacement. **Access level** Owner **What this function can do** For the proxy owner: require non-zero; emit ProxyOwnershipTransferred; set the new owner. **What this function cannot/should not do** Two-step transfer; transfer to zero; be called by a non-owner.

Contract Name	Function	Category
TokenProxy.sol	`upgradeTo(address newImplementation)`	**Main invariant(s)** The slot holds an authority that resolved to a non-zero implementation at upgrade time; the event reports the authority, not the implementation. **Access level** Owner **What this function can do** Despite the parameter name, for the proxy owner: require the new address is non-zero, differs from the stored authority, and is an ImplementationAuthority whose getImplementation() returns a non-zero address; store it as the authority; emit Upgraded with the authority address. **What this function cannot/should not do** Accept a raw implementation (it has no getImplementation() and reverts); accept an authority that currently resolves to zero; validate future compatibility or storage layout; be called by a non-owner; reject an authority whose implementation later changes to something malicious.
TokenProxy.sol	`upgradeToAndCall( address newImplementation, bytes memory data)`	**Main invariant(s)** Authority change and initialization call are atomic. **Access level** Owner **What this function can do** For the proxy owner: set the new authority, then self-call data with msg.value; require success; roll back the upgrade on failure. **What this function cannot/should not do** Validate the payload; execute a delegated payload while maintenance is enabled (the self-call is a non-owner call and reverts, although direct proxy functions remain reachable); recover ETH; be called by a non-owner.

Contract Name	Function	Category
TokenProxy.sol	`implementation()`	**Main invariant(s)** The returned implementation is whatever the current authority reports. **Access level** Public **What this function can do** Read the authority slot; if zero, return zero; otherwise call getImplementation() on it and return the result. **What this function cannot/should not do** Validate that the returned implementation has code; return a cached address; work if the authority reverts or returns malformed data (the call reverts).
TokenProxy.sol	`fallback()`	**Main invariant(s)** All token logic is reached through the authority's current implementation. **Access level** Public (Public; maintenance-restricted for non-owners.) **What this function can do** Delegatecall the dynamically resolved implementation with the full calldata, subject to the maintenance check. **What this function cannot/should not do** Bypass maintenance for non-owners; handle a zero implementation (it reverts); validate selectors; recover ETH.

Contract Name	Function	Category
TokenProxy.sol	`receive()`	**Main invariant(s)** No ETH custody path exists in the token implementation. **Access level** Public **What this function can do** Invoke _fallback for empty calldata. **What this function cannot/should not do** Accept plain ETH because Token has no receive/fallback.

3. Asset Flow Mapping (Critical Contracts)

This section identifies **where money actually lives** and **how it moves**.

3.1 Asset-Holding Contracts

List only contracts that **custody value**.

Contract	Asset Type	Custodied Assets
TradevuEscrow (behind Escrow Proxy)	USDC ERC-20	All investor principal and investment fees in orderPool[token]; all redemption money in settlementPool[token]; any unallocated surplus. The contract holds one USDC balance; totalEscrowed is the ledger claim on that balance.

3.2 Asset Entry & Exit Functions

This table maps **money movement**, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Tradevu Escrow	`placeOrder`	USDC amount + investmentFee from the investor	–	Public	Fee-on-transfer means the booked amount exceeds the received amount; actionID is caller-chosen and globally scoped; pending orders do not reserve the supply cap, so the offering can be oversubscribed and allocation depends on the issuer's resolution order; maxInvestment is per order, not per investor; there is no investor cancel; a blacklisted investor can revert their own transfer, not the issuer's.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Tradevu Escrow	`resolveOrder`	–	USDC principal to the issuer (msg.sender); investment fee to adminWallet; bill tokens minted to investor	Role holder	Principal is paid before redemption is funded, so the issuer is unsecured credit; fee, price, decimals, and supply are re-read at resolution; one failed transfer or mint reverts the whole batch; a blacklisted issuer blocks resolution, and a blacklisted adminWallet blocks it when a fee is due; minted supply is checked against the latched cap. Caller note: Token ISSUE_ROLE
Tradevu Escrow	`rejectOrder`	–	USDC amount + investmentFee refunded to the investor	Role holder	This is the only refund route and it has no deadline; the issuer must act; one blacklisted investor reverts the whole refund batch; the fee is re-read at rejection, so a token-logic fee change changes the refund. Caller note: Token ISSUE_ROLE

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Tradevu Escrow	`depositUSDC`	USDC exact gross deposit from the issuer	USDC distribution fee to adminWallet	Role holder	No deadline after maturity; the exact-amount rule prevents underfunding but not indefinite delay; fee-on-transfer makes the pool overstate its balance; a blacklisted adminWallet blocks funding; an oversized claimWindow can overflow the claimEndTime addition; the amount is derived from live minted supply; between maturity and funding it is fixed only while the escrow is the sole CONTROLLER_ROLE (the escrow's own burn path requires claimStarted, but Token.burn itself only requires maturity). Caller note: Token ISSUE_ROLE
Tradevu Escrow	`redeemBillTokens`	Bill tokens burned from the holder	USDC net share to the holder	Public	First-come-first-served against a finite pool; per-holder fee rounding can leave the tail short by dust that grows with the number of redeemers; a gross share of zero burns the whole balance for zero payout; a pool-capped payout burns only the proportional portion; the claim window is wall-clock and final; a blacklisted holder can only block their own redemption.
Tradevu Escrow	`withdrawRemainingUSDC`		USDC remaining in settlementPool to adminWallet	Role holder	Unclaimed holder funds are swept to adminWallet, not returned to holders or the issuer; the sweep must be strictly after claimEndTime; a zero remainder reverts and the offering can never be marked settled; a blacklisted adminWallet blocks the sweep. Caller note: Token ISSUE_ROLE

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Token	`mint`	–	Bill tokens to the investor	Role holder	No supply cap in the token itself; the escrow enforces the cap and the offering window; any address granted CONTROLLER_ROLE can mint within the window; a zero amount reverts, a zero recipient reverts. Caller note: Token CONTROLLER_ROLE (expected: escrow)
Token	`burn`	Bill tokens from the holder (destroyed)	–	Role holder	No allowance and no holder consent; any CONTROLLER_ROLE holder can burn any holder after maturity; burning without redemption leaves the corresponding USDC in the pool for the issuer's sweep to adminWallet. Caller note: Token CONTROLLER_ROLE (expected: escrow)
Token	`transfer / transferFrom`	Bill tokens from the sender	Bill tokens to the recipient	Public	The redemption claim moves with the token; there is no snapshot and no transfer restriction; a buyer must ensure the seller does not redeem first; NotTransferrable is declared but unused. Caller note: Holder or approved spender
Token	`approve`	- (allowance)	- (allowance)	Public	A spender can move the holder's redemption claim with transferFrom; standard ERC-20 allowance risk. Caller note: Holder

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Tradevu. We performed our audit according to the procedure described above.

3 Medium, 11 Low, 5 Informational severity issues were found, Which the Tradevu Team has Fixed

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With 8+ years of expertise, we've secured 1500+ projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey



AUDIT REPORT

November 2026

For

Tradevu

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com